

Assignment-10 (Lab-10.2)

Course : AI Assisted Coding

Topic:- Code Review and Quality: Using AI to Improve Code Quality and Readability

Name : Batti Chandan singh

Hall Ticket No : 2303A51515

Batch : 22

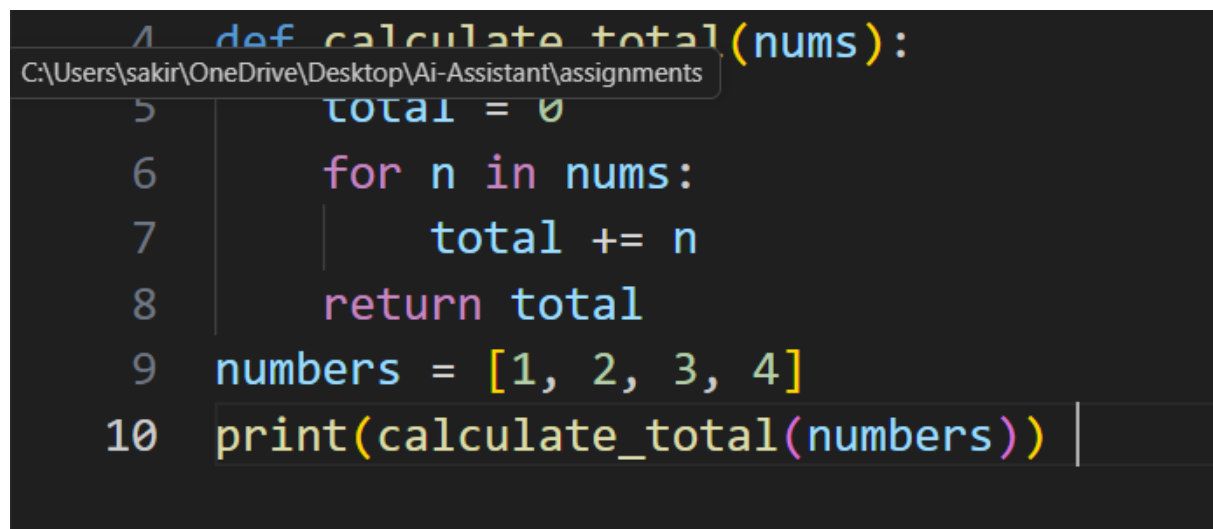
Task-01

Final Optimal Prompt:

Analyze the following Python code and correct all syntax and logical errors.
Provide the corrected executable Python code and briefly explain each error you fixed.
Code:

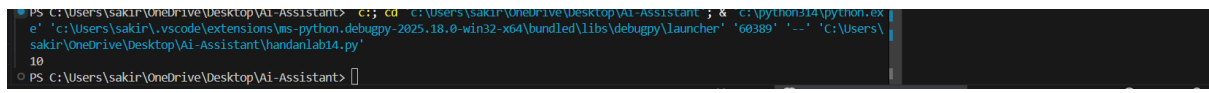
```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Code Screenshot :



```
4 def calculate_total(nums):
5     total = 0
6     for n in nums:
7         total += n
8     return total
9 numbers = [1, 2, 3, 4]
10 print(calculate_total(numbers))
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\AI-Assistant> cd c:\Users\sakir\OneDrive\Desktop\AI-Assistant ; & c:\python314\python.exe c:\Users\sakir\OneDrive\Desktop\AI-Assistant\handanlab14.py
10
PS C:\Users\sakir\OneDrive\Desktop\AI-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The original script contained multiple syntax and logical errors. The function definition missed a colon, and the loop statement also lacked a colon. The variable `sum` was used but the function returned `total`, which caused a logical error. Additionally, using `sum` as a variable name is not recommended because it overrides Python's built-in function. The corrected code replaces it with `total` and ensures proper indentation and syntax. A sample list was added to demonstrate functionality. The revised script now executes successfully and returns the correct result. These corrections improve reliability, maintainability, and compliance with Python coding standards.

Task-02

Final Optimal Prompt:

Refactor the following Python code so that it follows standard Python coding style guidelines (PEP 8).

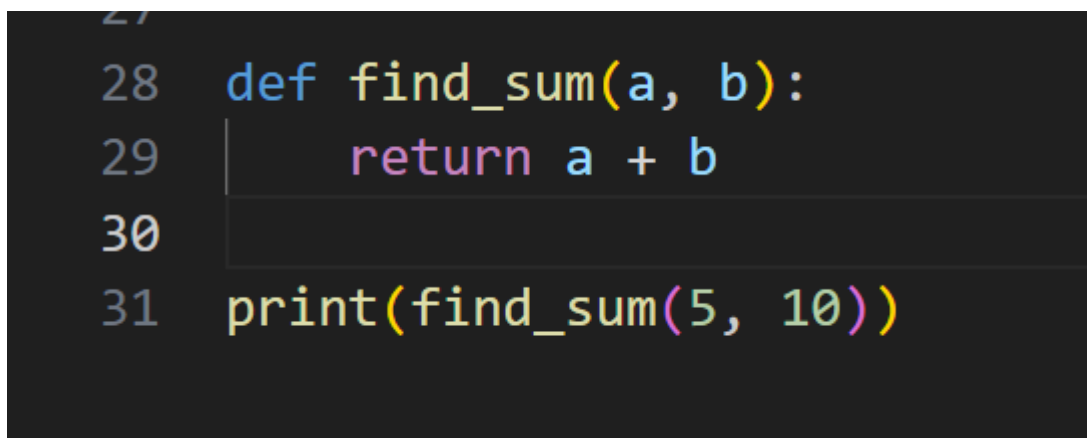
Ensure proper formatting, indentation, spacing, and readability while keeping the functionality unchanged.

Code:

```
def findSum(a,b):return a+b

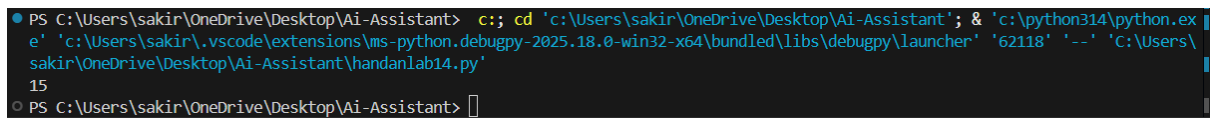
print(findSum(5,10))
```

Code Screenshot :



```
27
28 def find_sum(a, b):
29     return a + b
30
31 print(find_sum(5, 10))
```

Output Screenshot:

A terminal window showing a PowerShell command being executed. The command is: `c:; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62118' '--' 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant\handanlab14.py'`. The output shows the file path and the number 15.

```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62118' '--' 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant\handanlab14.py'
15
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The original code did not follow Python's recommended style guidelines. The function name used camelCase instead of the PEP 8 preferred snake_case format. The code was also written on a single line, reducing readability and maintainability. The refactored version separates statements properly, improves indentation, and renames the function to find_sum. Additional spacing and variable naming enhance clarity. These changes do not modify the program's logic but significantly improve code readability and consistency. Following style conventions ensures that the code is easier for other developers to understand and maintain, especially in collaborative projects.

Task-03

Final Optimal Prompt:

Improve the readability and clarity of the following Python code without changing its functionality.

Use meaningful function and variable names, proper indentation, and clear structure.

Code:

```
python
def f(x,y):
return x-y*2
print(f(10,3))
```

Code Screenshot :

```

32
33 ~ def calculate_result(number, multiplier):
34     result = number - multiplier * 2
35     return result
36
37
38 print(calculate_result(50, 3))

```

Output Screenshot:

```

4
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '63295' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\handanlab14.py'
44
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The original function used very short variable names such as f, x, and y, which made the program difficult to understand. To improve readability, meaningful names were introduced for both the function and parameters. The new function name calculate_result better reflects the purpose of the calculation. Proper spacing and indentation were also applied to align with Python coding conventions. These changes improve maintainability because other developers can quickly understand what the function does without analyzing the logic deeply. The functionality remains exactly the same as the original code while significantly improving clarity and structure.

Task-04

Final Optimal Prompt:

Refactor the following Python code to eliminate repetition by creating reusable functions. Ensure the code is modular and maintainable.

Code:

```

print("Hello Ram")
print("Hello Sita")
print("Hello Ravi")

```

Code Screenshot :

```
55 def greet_person(name):  
56     print(f"Hello {name}")  
57  
58  
59 greet_person("Ram")  
60 greet_person("Sita")  
61 greet_person("Ravi")
```

Output Screenshot:

```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c;; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63258' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\handanlab14.py'  
Hello Ram  
Hello Sita  
Hello Ravi
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The original program contained repetitive print statements, which is not efficient or maintainable. If more names were added, the code would require additional lines for each greeting. The refactored solution introduces a reusable function named `greet`, which accepts a name parameter and prints the greeting message. A list of names is then used with a loop to call the function repeatedly. This approach eliminates duplication and follows the DRY (Don't Repeat Yourself) principle. The code becomes shorter, cleaner, and easier to modify. If the greeting format needs to change later, it can be updated in one place instead of multiple lines.

Tack-05

Final Optimal Prompt:

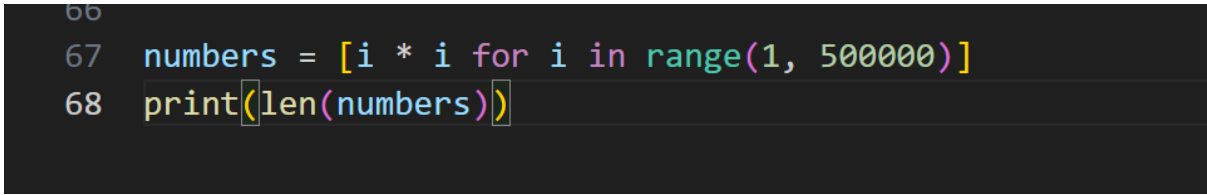
Optimize the following Python code to improve performance while keeping the output the same.

Use more efficient Python constructs if possible and briefly explain the optimization.

Code:

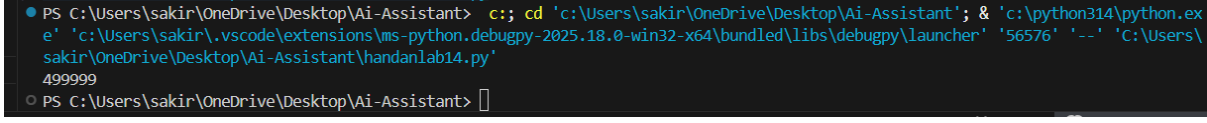
```
python
numbers = [ ]
for i in range(1, 500000):
    numbers.append(i * i)
print(len(numbers))
```

Code Screenshot :



```
66
67 numbers = [i * i for i in range(1, 500000)]
68 print(len(numbers))
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '56576' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\handanlab14.py'
499999
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The original program used a loop to append squared values to a list, which required multiple lines of code and repeated method calls. The optimized version replaces this loop with a list comprehension. List comprehensions are generally faster and more concise because they are implemented internally in Python with optimized performance. This reduces the number of operations and improves readability. The resulting list still contains the square of each number from 1 to 499999, maintaining the same functionality as the original code. The final statement prints the length of the list, confirming that the result remains correct while the code becomes more efficient and compact.